

ASU - AI for Business: Creating Smart Business Negotiation Bots

Our goal for this project is to build a local, open-source LLM negotiation agent that performs robustly across multiple negotiation settings, including task allocation and buyer-seller bargaining. We use the datasets, evaluation framework, and findings from [Are LLMs Effective Negotiators?](#) and [Measuring Bargaining Abilities of LLMs](#) as our primary benchmark, since it provides a structured, capability-level decomposition of negotiation performance across models and tasks.

Reinforcement Learning Algorithms:

DPO - Direct Preference Optimization

Summary

Direct Policy Optimization is a RL method that uses a policy model (in our case this would be a LLM of choice) and a reward model. The method is set up by having an input for the policy model, in our case this would be a chain of negation or some string input for the model, and two possible outcomes/outputs of the policy model given the input. Both of the outputs in the output pair will be given a utility or reward (this may be done by some utility function, LLM as judge, or human annotation). One output in the pair will have a higher reward value than the other, making it the preferred output. Using a data set of these (input, ((output1, reward), (output2, reward))) tuples a reward model, r , is trained. Our goal is to have the policy, LLM, to output outputs with the highest reward value.

Bradley-Terry Model

The Bradley-terry (bt) model turns rewards into probabilities. This is used to turn the reward in the loss function on which the model is trained (tuned) with, into a probability. We need a model to turn reward values into a probability that extended more than just $\text{reward1}/(\text{reward1} + \text{reward2})$ and $\text{reward2}/(\text{reward1} + \text{reward2})$, this is due to the possibility of negative rewards. So we allow the probabilities to be the sigmoid of the reward minus the second output's reward $\text{sigmoid}(\text{the difference in rewards})$.

KL Divergence

KL Divergence is used in the loss function on which the model is trained (tuned) with, and it aims to calculate the difference between one model's output distribution and another model's

output distribution. This number is used to make sure that the model is not updated too drastically at one tuning step.

Thoughts On DPO for Our Use Case

For starters, we would need a dataset to use for the DPO tuning. Which means we need a dataset that has chains of negotiations (we don't want to just tune our model on the opening greeting of a negotiation, so we would train on a variety of points within a negotiation chain, for many negotiation chains) along with the two possible outputs that our model would give. For all of our reviewed datasets we only have one output and not a pair of outputs, this means we would have to generate new data for our datasets.

Next, we would have to give each output pair in the dataset a reward value. The team would have to do human annotations on each of these outputs or have an LLM-as a judge provide reward values. This first option seems time consuming and I would be wary of the team's ability to accurately give a reward value without formulating a strict and rigorous grading scheme as we are not linguists and human negotiation is not our domain of focus. For the second option we would have to use a high performing model such as GPT-4 or 5 which could be expensive for a large training set, and at that point we may be training our model to act more like GPT.

Finally, I personally don't like this method because it is similar to SFT in the sense that we don't allow the model to explore new strategies and negotiation paths. We only present output options at each training point, and do not focus on either profit/deal success reward and strategic reward/classification.

PPO - Proximal Policy Optimization

Summary

PPO isn't a reinforcement learning method in itself, but instead an optimization algorithm used to update the policy (the LLM in our case) based on the observed rewards within reinforcement learning. PPO can be used in different RL settings such as Reinforcement Learning w/ Human Feedback (RLHF) and Environment-Based Reinforcement Learning. Alternatives to PPO include DPO and GRPO. PPO aims to update the policy LLM very stably and subtly to not ruin the response distribution quickly.

Value Neural Network

To update the policy or LLM based on an observed output in reinforcement learning we need to first score the output based on the reward model and also calculate the difference between that reward and the average reward that is experienced from an output given the input. This basically means we need a model to tell us what the average reward is for given input so that we can determine if the output observed was good or bad. Based on if the observed output was

good or bad, we would update the policy to reflect that. The value network is then trained before reinforcement learning is started by calculating the reward of say the base model's output over a series of inputs, and training a neural network to predict the reward for a given input. This value model is also updated along with the policy during reinforcement learning to reflect the updates to the model.

Thoughts on PPO for our use case

PPO allows for the model to explore more strategy paths as opposed to DPO which limits the model exploration to "good" and "bad" output, and PPO can end up discovering strategies never explicitly expressed through the training data. PPO can be used in an environment based reinforcement learning setup which I think is good.

There is a need to train the value model and intertwine it and its training into the policy updates. This is more work to set up, and there are some things to think about such as how will we design the value, and what libraries are available for this.

GRPO - Group Relative Policy Optimization

Summary

GRPO is similar to PPO as an optimization algorithm, but lacks the need for a value model. Instead of a neural network based value model that is used to predict the average reward value for a given input to the policy (LLM), an average reward is found by taking the average of the reward values of a sample of outputs for a given input. This means that for a given training point (input) the model is queried multiple times, let's say 8. The reward is calculated for each of the 8 outputs, and the average is taken between them which is seen as the "expected reward" for the input. The policy is updated based on the differences between each of the 8 outputs and the "expected reward". The LLM distribution is skewed to favor responses of the outputs with reward greater than the "expected reward" (the preferred outputs), and skewed to disfavor the responses of the outputs with reward less than the "expected reward".

Policy Update

Updating the policy for GRPO is similar to PPO but of course some of the computation changes as we now consider multiple reward values instead of a value model reward and actual reward value. GRPO is also more sample-efficient as the optimization samples x (in our previous example it was 8) outputs per training point, which can provide more efficient policy updates.

Personal Thoughts

I think GRPO is a solid option as far as optimization method goes. It loses the need for a trained value model, it can be more efficient than PPO, it promotes exploration, and can be implemented with self-play and environment based learning. Due to the sole reason of having

more exploration potential than DPO I think this optimization policy should be selected over DPO. While I think PPO could be a solid choice for our use-case I think the need for a value model is overhead in design and development that could affect the timeline of our project. GRPO is newer than PPO and has proven incredibly effective as seen with the Deepseek R1 model.

Reward Sources / Setting Design:

Human Feedback

In Reinforcement Learning with Human Feedback (RLHF) the reward model is dependent on human annotation or preferences. For each training input, an x number of responses are reviewed by a human annotator who scores or marks a preferred response which is then used within the selected optimization policy (PPO, DPO, GRPO). The setup for reward annotation and optimization is of course dependent on the optimization policy, but the underlying theme of RLHF is that the reward model is based on human annotation. This means it is beneficial to have human annotators be field experts in the use-case (target task) domain to help with annotation. Human annotation is both expensive and slow compared to other options, because the reward is not calculated from the environment or a logic/code based reward model. A second problem arises with human annotation, which is each annotator may have different views on what constitutes a response as a certain score or more preferred, which can introduce bias into the reinforcement learning process.

Environment Based

Environment based RL is surrounded by a build environment or simulation in which the to-be trained agent interacts with. This environment and the actions that the agent takes within the environment provides the source of the reward calculation. For example in the scope of our use-case, the environment can be similar to the game and game rules defined in [Measuring Bargaining Abilities of LLMs](#) where an agent has a finite set of actions to take at each step of the game. Because a reward model gives a value to the output of a training input based on the environment and agent actions a reward can be easily derived logically through code based evaluation or human evaluation (there can be some crossover in reward/source RL setups). An example of this logically computed reward could be the normalized profit that is an outcome of the negotiation or a negative value if no deal is made. Environment based RL can make it easy and efficient to design a reward model. It is important to note that the reward model can introduce bias depending on what is taken into account for the final reward score, which is why it is important to spend time designing it to capture all performance metrics that affect the

performance of the agent's goal task. This setup can easily be scaled to more training data as the environment does not change.

Self-play

Self-Play RL is similar to environment based RL in the sense that the agent plays in a defined environment or task setup and the reward model can be derived from the agent's actions and performance within the environment, but there is a large difference that constitutes self-play. In self-play setups the agent plays against itself, the same policy model. This means that the environment from the perspective of either model is non-deterministic since the other player (policy) is constantly being updated via the selected policy optimization model. In this setting the two models co-evolve due to a reward being calculated on each player's behalf and an update being made to the policy. The policy is now training on data that is not defined in a "training set" because it is generated from the other model within each training run. Due to the training data constantly evolving as the model is updated, techniques and tactics can be learned that were never shown in a traditional "training set". This can lead to tactics emerging that are unexpected, but it can also cause both models to end up pigeon-holding each other into a very specific strategy or tactics also known as "strategy collapse".

Methods for Using Optimization Algorithms in Chained Negotiation Environments

How can we apply the reviewed policy optimization algorithms to multi-turn (chained) negotiations? In a multi-turn negotiation we only know the true outcome of the negotiation after multiple iterations of communication or (input, output) pairs. So how do we update the policy for each output in a negotiation?

Terminal Rewards

One method is to calculate the terminal reward based on the end outcome of the negotiation; this may be derived from the binary deal or no-deal value, or a terminal profit value. Using the terminal reward, each input -> output pair is given the same reward value as the terminal reward. This is the most straightforward and simple to implement reward method, but it does have a chance of miss representing the earlier negotiation responses as each response is given the same reward as the final response or terminal reward, while there could have been a previous response that greatly affected the negotiation in a positive way and another previous response that greatly affected the negotiation in a negative way, causing error in the response evaluation.

Discounted Rewards

Discounted rewards are similar to terminal rewards in the sense that each input -> output turn in the negotiation is based on the terminal reward, but the difference here is that at each input -> output other than the terminal response a discount is applied to the reward, which increases the further away the input -> output pair is from the terminal pair. For example we can apply a discount of 0.1% at each step moving away from the terminal reward, e.g. if the terminal reward is 1, the second to last response would receive a reward value of 0.99 ($1 * 0.9^1$), the third to last response a reward value of 0.891 ($1 * 0.9^2$), and so on. This reflects the idea that the final response (or output) is the most important and to not drastically change the policy in the scope of the mid to early negotiation steps which may be mis-represented due to the terminal reward.

Intermediate Rewards

This reward technique aims to solve this mis-representation of early to mid negotiation responses due to the terminal reward, through intermediate reward values. An intermediate reward value is calculated at each intermediate negotiation step and represents how effective that response was in the scope of just that step, disregarding the terminal value. For example intermediate rewards can be given for the response type (offer, counter offer, etc.) or the models understanding at that point of the negotiation (C1 - C4 outlined in <https://aclanthology.org/2024.findings-emnlp.310.pdf>). This intermediate reward can be used directly with the policy optimization algorithm or it can be added to the terminal reward to be used in the policy optimization algorithm. This reward method gives the most focus to the performance of the model at each negotiation step, but is also the most difficult to implement, as a reward model needs to be defined outside of the terminal reward model.

Three RL Design Ideas

Environment Based Self Play - GRPO - Terminal Reward

Optimization Policy: GRPO

Training Framework: Environment Based

Reward Signal: Terminal Reward

Summary:

This design aims to combine simplicity and effectiveness. GRPO is selected to eliminate the need for a value model, while also allowing the model to explore new techniques and strategies as opposed to DPO. An environment based + self play training framework is selected to eliminate the need for human intervention and provides an easy logic based/computable reward

function, because of this we can easily train the model with a high number of training points. A terminal reward signal is chosen as it is derived completely from the environment and requires no extra reward model as would an intermediate reward signal. This design aims for quick development and hands off training

Training Environment:

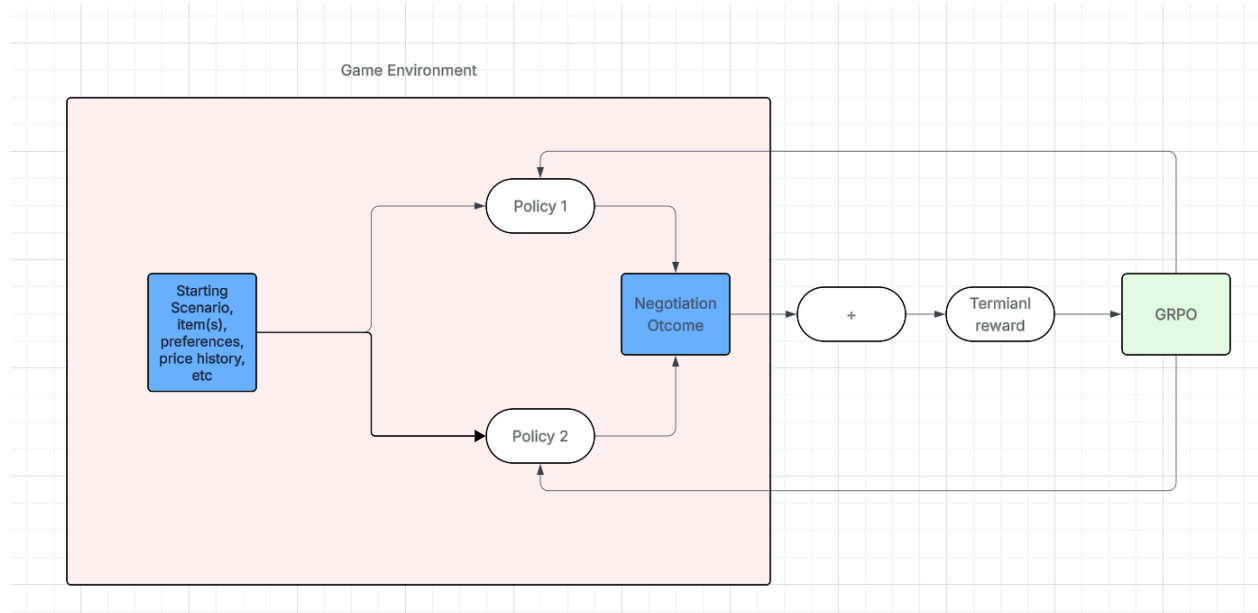
The training environment will follow a strict turn based environment based on one or multiple of the datasets mentioned below. An example can be shown for training on the CaSiNo Dataset, which includes 1030 starting scenarios including player preferences and values for each resource for each player:

Preferences & Arguments	
P1	P2
<i>High: Water: We like to go on runs and it increases the need of this.</i>	<i>High: Food: Food really increases everyones morale.</i>
<i>Medium: Food: Food overall is a good mood booster.</i>	<i>Medium: Firewood: We like to have a large fire.</i>
<i>Low: Firewood: We do not care for fire and it is not necessary to us.</i>	<i>Low: Water: We don't drink water that often.</i>

The starting scenario is the only thing used from the dataset. The environment rules are designed similar to the negotiation environment outlined in the [amazon item negotiation Measuring Bargaining Abilities of LLMs](#) where each agent has a set number of actions and there is a max number of negotiation steps each agent can take. The terminal reward is a function of the end result points, max number of points per one player, and max number of points if both players are fully pros-social (play with each other in mind). While in the CaSiNo dataset there is a limited number of starting scenarios, preferences, the number of items and preference classifications can be extended to create more training examples.

This setup can also be ran with a scraped amazon dataset similar to the CamelCamel dataset from [Measuring Bargaining Abilities of LLMs](#) and the negotiation environment can be the same as outlined in the [Measuring Bargaining Abilities of LLMs](#) paper as well. Here a large number of training points can be scraped, and the code for the game setup can be reused. In this example the terminal reward can be a function of normalized profits as explained in the paper.

Diagram:



Additional Notes:

Be weary of strategy collapse. To mitigate this we can introduce frozen model play.

Environment Based Self-play - GRPO - Terminal Reward + Intermediate Reward

Optimization Policy: GRPO

Training Framework: Environment Based + Self Play

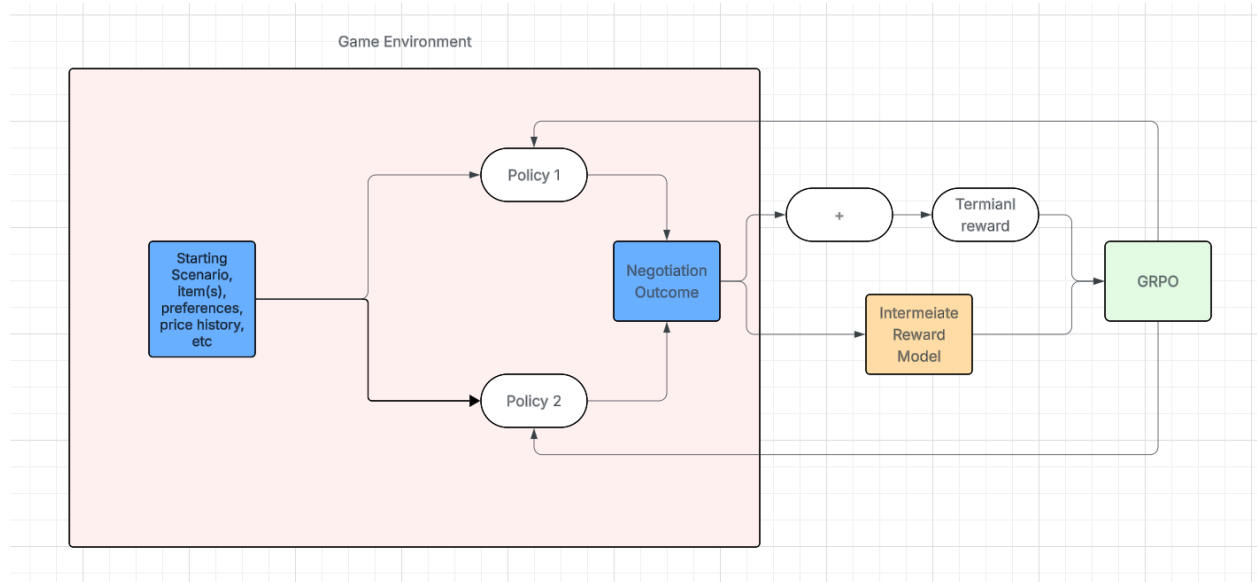
Reward Signal: Terminal Reward + Intermediate Reward

Summary:

This design aims to build off the previously mentioned design by adding an Intermediate Reward model to better represent the quality of each step in the negotiation process as opposed to just the terminal reward. The setup and game environment can be copied from the previous example using either an already existing dataset such as CaSiNo or a newly scraped amazon dataset, using the corresponding game environment. As mentioned the main difference is the intermediate reward model, which can be extended from the [Are LLMs Effective Negotiators?](#) Paper which aims to quantify an LLM's ability to negotiate with four benchmarks that make up the overall goal of "understand the negotiation context and outcome" ([Are LLMs Effective Negotiators?](#)). This will require the team to replicate (no github provided) the evaluation

framework which is shared in this paper. The team can simplify this process by selecting only one or two important benchmarks to make up the reward model.

Diagram:



Environment Based - GRPO - Intermediate Reward From <https://arxiv.org/pdf/2402.13550>

Optimization Policy: GRPO

Training Framework: Environment Based

Reward Signal: Intermediate Reward (Perform optimization step at each step of negotiation)

Summary:

This design aims to focus on the ability of an LLM to comprehend the negotiation context, infer partner preference and strategy, and interpret strategies throughout the negotiation process as opposed to the final terminal reward of the negotiation. It is thought that if we can train a model based on the before mentioned tasks that the performance of negotiation outcome will increase as well. This is done by using the framework outlined in [Are LLMs Effective Negotiators?](#) as the reward model at each turn of a negotiation. This design follows a different structure than the other two proposed designs, as the RL loop does not wait until a complete dialog of negotiation

is finished until running the optimization policy. This is because a terminal reward (negotiation outcome) is not used for the optimization policy.